



Rapport de projet : PC3R

Abdullah AL MAMUN (21124650)

Yahya Mudallal (21238547)

Mai 2026

Table des matières

Table des matières	<u>2</u>
Introduction et Présentation du projet	<u>3</u>
Conception et Architecture Générale	<u>4</u>
1. Schéma général	<u>4</u>
2. Base de donnée	<u>4</u>
3. API REST	<u>6</u>
Implémentation du Frontend	<u>8</u>
Implémentation du Backend	<u>9</u>
Déploiement	<u>10</u>
Utilisation de L'IA	<u>11</u>
Conclusion	<u>11</u>

Introduction et Présentation du projet

Ce projet a été réalisé dans le cadre de l'unité d'enseignement Programmation Concurrente, Réactive, Répartie et Réticulaire (PC3R), dont l'objectif était de concevoir une application Web sur un sujet libre. Notre choix s'est porté sur le développement de **Briefly**, une plateforme d'agrégation d'articles de presse internationale. L'application permet aux utilisateurs de consulter des actualités, de voter pour les articles via un système d'upvote et de downvote, et de commenter des articles. L'une des fonctionnalités clés de Briefly est l'intégration d'une Intelligence Artificielle pour générer automatiquement des résumés concis des articles. Le flux de données est entièrement automatisé : les articles sont récupérés quotidiennement à 2h00 du matin via l'API [NewsData.io](https://newsdata.io/), qui permet d'accéder à des actualités mondiales. Pour ce projet, la collecte se concentre exclusivement sur les articles en langue anglaise. Les résumés automatiques sont quant à eux générés en sollicitant l'API Gemini via la plateforme [Google AI Studio](https://aistudio.google.com/).

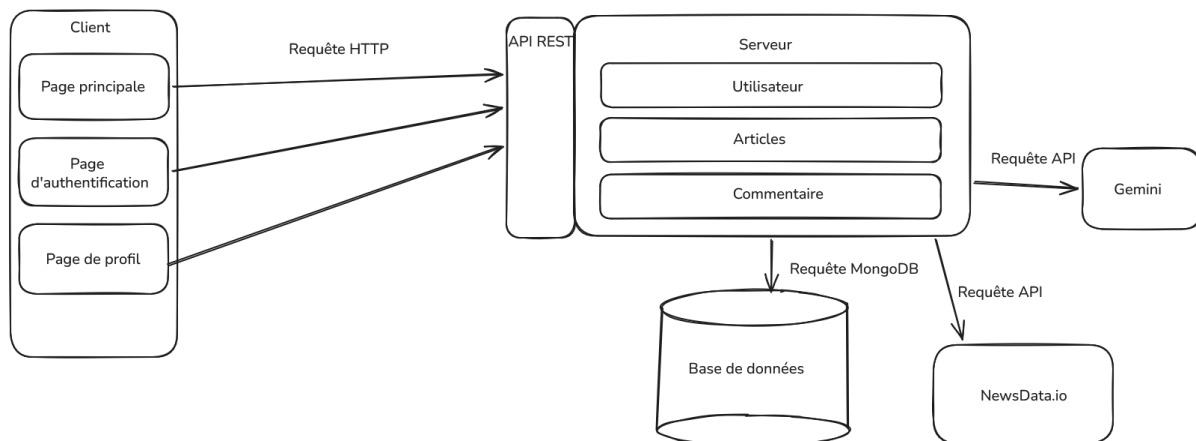
Concernant le parcours utilisateur, le visiteur accède lors de sa première visite à la page principale qui présente un flux d'actualités défilant. Chaque article affiche un titre, sa source et, éventuellement, une description. Un clic sur le titre redirige le visiteur vers le site d'origine de l'article. Bien que la lecture des commentaires soit accessible à tous, certaines actions interactives requièrent une authentification. Ainsi, si un utilisateur non connecté tente de publier un commentaire ou de voter pour un article, il est automatiquement redirigé vers l'interface de connexion. Cette page propose un formulaire d'authentification classique ainsi qu'une option d'inscription pour la création d'un nouveau compte.

Pour la partie cliente, le développement s'est fait en TypeScript avec la bibliothèque React afin de construire une interface graphique dynamique et réactive. Côté serveur, l'API REST est développée intégralement en Go, aucun framework de haut niveau n'a été introduit, l'ensemble reposant uniquement sur le package natif net/http. MongoDB a été utilisé pour la base de données. Enfin, l'infrastructure s'appuie sur la plateforme [Render](https://render.com/) pour l'hébergement du client et du serveur, tandis que la base de données est hébergée par le service cloud [MongoDB Atlas](https://cloud.mongodb.com/). La planification de la tâche quotidienne de récupération des articles est confiée à cron-job.org, un choix technique motivé par des contraintes qui seront détaillées dans la section dédiée au déploiement.

Conception et Architecture Générale

1. Schéma général

L'application est structurée autour d'une architecture découplée, composée d'une partie client développée en TypeScript avec React et d'une API REST offerte par un serveur en Golang. Ce serveur centralise la logique métier et interagit de manière asynchrone avec la base de données MongoDB, ainsi qu'avec les API externes de [NewsData.io](https://newsdata.io/) pour la récupération des flux d'actualités et de [Google AI Studio](https://ai.google.dev/gemini-api) (Gemini) pour la génération des résumés. Le schéma suivant illustre les divers composants de cette architecture ainsi que leurs interactions.



2. Base de données

La base de données NoSQL MongoDB s'organise autour de quatre collections principales : les utilisateurs, les articles, les commentaires et les votes. Chaque tableau ci-dessous détaille rigoureusement la liste des attributs, leur type de données ainsi qu'une description.

Document : users (utilisateurs)

Champ	Type	Description
_id	ObjectId	Identifiant unique pour chaque utilisateur.
email	String	Email de l'utilisateur.
first_name	String	Prénom de l'utilisateur.
last_name	String	Nom de famille de l'utilisateur.

password	String	Empreinte sécurisée du mot de passe de l'utilisateur.
created_at	Date	Date de création du compte.
updated_at	Date	Date de dernière modification du compte.
is_admin	Boolean	Drapeau booléen définissant les privilèges.

Document : articles

Champ	Type	Description
_id	ObjectId	Identifiant unique de l'article.
title	String	Titre de l'article.
description	String	Description de l'article.
original_url	String	URL de l'article.
image_url	String	URL de l'image de l'article.
source	String	Source de l'article.
up_votes	Integer	Nombre de votes positifs de l'article.
down_votes	Integer	Nombre de votes négatifs de l'article.
nb_comments	Integer	Nombre de commentaires sur l'article.
summary	String	Résumé de l'article.
published_at	Date	Date de publication de l'article.
created_at	Date	Date de création du document.
updated_at	Date	Date de dernière modification du document.

Document : comments (Commentaires)

Champ	Type	Description
_id	ObjectId	Identifiant unique du commentaire.
author_id	ObjectId	Identifiant de l'auteur du commentaire.
author_name	String	Nom et prénom de l'auteur du commentaire.
article_id	ObjectId	Identifiant de l'article commenté
content	String	Contenue du commentaire

Champ	Type	Description
parent_id	ObjectID	Identifiant du commentaire parent. (peut-être nul)
created_at	Date	Date de création du commentaire.
updated_at	Date	Date de dernière modification du commentaire.

Document : votes

Champ	Type	Description / Exemple
_id	ObjectId	Identifiant du vote.
article_id	ObjectId	Identifiant de l'article voté.
user_id	ObjectId	Identifiant de l'utilisateur votant.
type	Integer	Type de vote (positif : 1, négatif : -1).

3. API REST

Méthode et Route	Description et Contraintes
GET /v1/health	Renvoie un statut 200 OK permettant de vérifier l'état de fonctionnement et la disponibilité du serveur.
GET /v1/articles	Renvoie une liste d'articles paginée en fonction des paramètres de requête page et limit passés dans l'URL.
GET /v1/articles/{id}	Renvoie les détails complets de l'article correspondant à l'identifiant unique fourni.
GET /v1/comments/{id}	Renvoie le commentaire correspondant à l'identifiant spécifié au format JSON.
GET /v1/comments/article/{id}	Récupère l'intégralité des commentaires et des réponses associés à l'article ciblé.
GET /v1/users/{id}	Renvoie les informations publiques du profil de l'utilisateur correspondant à l'identifiant.

Méthode et Route	Description et Contraintes
POST /v1/users	Crée un nouveau compte utilisateur après validation du format de l'email et de la complexité du mot de passe. Renvoie un token JWT en cas de succès.
POST /v1/login	Authentifie l'utilisateur si les identifiants fournis dans le corps de la requête sont valides, puis génère et renvoie un token JWT.
POST /v1/comments	Publie un nouveau commentaire lié à un article. Cette action nécessite un token JWT valide transmis dans l'en-tête de la requête.
POST /v1/articles/{id}/upvote	Enregistre un vote positif pour l'article ciblé. L'identité du votant est extraite de manière sécurisée depuis le token JWT.
POST /v1/articles/{id}/downvote	Enregistre un vote négatif pour l'article ciblé. L'identité du votant est également extraite depuis le token JWT.
POST /v1/articles/{id}/summary	Déclenche manuellement un appel asynchrone vers l'API Gemini de Google AI Studio pour générer le résumé de l'article. Requiert un token JWT.
POST /v1/sync-articles	Déclenche la synchronisation avec NewsData.io pour récupérer dix nouveaux articles. Cette route est protégée par une clé secrète dans l'en-tête.
DELETE /v1/comments/{id}	Supprime définitivement un commentaire. Cette action requiert un token JWT et n'est autorisée que pour l'auteur du message ou un administrateur.
DELETE /v1/articles/{id}	Supprime l'article spécifié de la base de données. Cette opération est strictement réservée aux utilisateurs disposant du rôle d'administrateur.
DELETE /v1/users/{id}	Supprime le compte d'un utilisateur de la plateforme. Cette opération est strictement réservée aux administrateurs.

Implémentation du Frontend

La partie client de l'application est développée en TypeScript en s'appuyant sur la bibliothèque React et l'outil de build Vite. L'intégration du module React Router permet de structurer le frontend autour de trois vues principales : la page d'accueil, la page d'authentification et la page de profil.

La page d'accueil, accessible par défaut via la route racine "/", constitue le point d'entrée de tout utilisateur. Cette vue est gérée par le composant `HomePage`, qui intègre un en-tête global via le composant `Header`. Ce dernier affiche le logo du site ainsi qu'un bouton dynamique permettant la connexion ou la déconnexion de l'utilisateur. Lorsqu'un utilisateur est authentifié, le `Header` affiche ses initiales, offrant un lien direct vers la page de profil. Le composant `HomePage` orchestre la récupération des données en interrogeant la route `GET /v1/articles` de l'API REST, puis délègue l'affichage de chaque actualité au composant `NewsCard`. Ce composant est responsable de la présentation des informations de l'article, des boutons d'interaction pour les votes, de l'affichage du fil de discussion et de la consultation du résumé. Si l'utilisateur vote pour un article, `NewsCard` émet une requête vers l'endpoint de vote dédié. De même, l'ouverture de l'espace de discussion déclenche une requête de récupération des commentaires de l'article. Enfin, pour l'affichage du résumé, le composant vérifie si celui-ci a déjà été généré par l'IA ; si ce n'est pas le cas, une requête est envoyée pour initier sa génération asynchrone.

La page d'authentification est associée à la route `/auth` et implémentée par le composant `AuthPage`. Ce dernier propose un formulaire unifié pour la connexion et l'inscription, transmettant les données aux endpoints correspondants du serveur. En cas de succès, les informations de l'utilisateur sont injectées dans un contexte React global (`UserContext`), et le jeton JWT est sauvegardé dans le `localStorage` du navigateur, assurant ainsi la persistance de la session pour les requêtes authentifiées.

La page de profil, configurée sur la route `/profile` via le composant `ProfilePage`, est protégée et accessible uniquement aux utilisateurs authentifiés. Elle extrait les données de la session depuis le `UserContext` pour les afficher à l'écran. Pour finir, la charte graphique de l'application est gérée de manière modulaire à l'aide de fichiers CSS dédiés à chaque partie, tandis que le typage strict des structures de données renvoyées par l'API est centralisé dans le fichier `types.tsx`.

Implémentation du Backend

Le serveur de l'application, développé en Go à l'aide du package natif `net/http`, expose les points d'entrée de notre API REST. Il est conçu selon une architecture logicielle découplée en trois couches distinctes :

La couche **Handlers** prend en charge la gestion directe des requêtes HTTP. Elle est responsable de la réception des requêtes, de l'extraction des données (depuis le corps de la requête ou les paramètres de l'URL), et de la sérialisation de la réponse au format JSON en cas de succès, ou du renvoi du code d'erreur HTTP approprié le cas échéant.

La couche **Services** encapsule la logique métier et les traitements algorithmiques internes du serveur, coupés de toute logique de transport (HTTP). C'est ici que sont orchestrées les vérifications de validité des données, comme la validation des formats d'adresses email ou le hachage sécurisé des mots de passe.

La couche **Repositories** gère exclusivement la persistance. Elle communique directement avec l'instance de la base de données MongoDB pour y exécuter les opérations de création, de lecture, de mise à jour et de suppression.

Chacune de ces couches est matérialisée dans le code par un package éponyme : `handlers`, `services` et `repositories`. Afin de respecter une séparation stricte des domaines, chacun de ces packages est divisé en trois fichiers (`articles.go`, `users.go` et `comments.go`) dédiés à la ressource correspondante. En parallèle, le package `models` centralise les structures de données de l'application. Ces structures intègrent des balises spécifiques (*struct tags*) permettant de mapper automatiquement les champs en BSON pour MongoDB et en JSON pour les échanges avec le client.

Le package `middleware` regroupe les intergiciels qui interceptent la requête HTTP en amont des handlers afin d'exécuter des vérifications ou des traitements transversaux. Les middlewares implémentés sont les suivants :

- `authMiddleware.go` : Intercepte les requêtes sur les routes protégées, décode et valide le token JWT présent dans l'en-tête, puis extrait l'identifiant de l'utilisateur pour le transmettre au contexte de la requête.
- `optionalAuthMiddleware.go` : Possède un fonctionnement similaire au middleware d'authentification, mais n'interrompt pas la

requête si le token est absent, permettant ainsi un accès en mode « invité ».

- `logger.go` : Assure la traçabilité en inscrivant dans la console du serveur les logs des requêtes reçues, incluant la méthode HTTP, la route, la date de réception et le temps de traitement global.
- `recover.go` : Agit comme une sécurité d'exécution en interceptant les éventuels cas de *panic* du serveur, évitant ainsi le crash du processus en renvoyant proprement une erreur HTTP 500 (Internal Server Error).
- `enableCORS.go` : Injecte les en-têtes HTTP nécessaires pour autoriser le partage de ressources cross-origin (CORS) avec l'URL du frontend React.

Enfin, le fichier `api.go`, situé dans le package `api`, centralise la configuration globale du serveur. Il déclare le routeur, associe chaque endpoint de l'API REST à son handler dédié et y applique la chaîne de middlewares requise. Ce module expose la fonction d'initialisation réseau qui est appelée par le point d'entrée principal de l'application (`main.go`) pour démarrer l'écoute du backend.

Déploiement

Pour la mise en ligne du projet Briefly, différents services cloud ont été mobilisés, principalement la plateforme PaaS [Render](#). Ce fournisseur permet d'héberger divers types d'applications comme des sites statiques, des services web ou des tâches planifiées. Le frontend de Briefly a été déployé en tant que site statique, tandis que le backend est hébergé en tant que service web autonome. Pour le déploiement du backend, un fichier Dockerfile a été conçu afin de standardiser l'environnement, de définir les étapes de compilation du code Go et de spécifier la commande de lancement du serveur. Le dépôt GitHub du projet ayant été directement lié aux services Render, un pipeline élémentaire d'intégration et de déploiement continu est actif : chaque nouveau commit poussé sur la branche principale déclenche automatiquement une recompilation et la mise en production de la dernière version en date.

L'utilisation de la formule gratuite de Render a toutefois imposé une contrainte technique majeure, puisque la plateforme met en veille le service web après quinze minutes d'inactivité sur l'API. Cette caractéristique posait problème pour notre fonctionnalité de synchronisation automatisée des articles. Pour réveiller le serveur et déclencher l'appel à l'endpoint de récupération des

actualités tous les jours à 2h00 du matin, une planification temporelle était indispensable. Les outils de cron internes de Render étant payants, la solution a consisté à externaliser cette tâche en utilisant la plateforme cron-job.org. Ce service tiers permet de programmer des requêtes HTTP régulières à intervalles définis, et a été configuré pour envoyer une requête POST sur notre route de synchronisation chaque nuit, résolvant ainsi le problème de mise en veille tout en automatisant la mise à jour des données.

En ce qui concerne la base de données, le choix s'est naturellement porté sur [MongoDB Atlas](https://cloud.mongodb.com), le service cloud officiel de MongoDB. L'offre gratuite propose un cluster partagé avec une capacité de stockage de 512 Mo. Cette mémoire est largement suffisante pour contenir les documents relatifs aux utilisateurs, aux commentaires et à l'historique des articles indexés pour notre projet, garantissant ainsi une haute disponibilité de nos données sans surcoût d'infrastructure.

Utilisation de l'IA

L'intelligence artificielle générative a été mobilisée à plusieurs étapes de la réalisation de ce projet, agissant comme un outil d'assistance au développement. Côté frontend, elle a été consultée pour optimiser l'interface graphique de certains composants React en générant des feuilles de style CSS spécifiques, et a également servi à concevoir les visuels et le logo de Briefly. Concernant l'architecture et le déploiement, l'IA a été une source d'information utile pour comparer les différents services cloud du marché et identifier les solutions gratuites les plus adaptées à nos contraintes de mise en veille, ce qui nous a orientés vers l'utilisation combinée de Render et de cron-job.org. Enfin, l'écriture et la configuration du fichier Dockerfile utilisé pour la conteneurisation et le déploiement du backend en Go ont également été finalisées avec l'appui de ces outils.

Conclusion

Le projet étant désormais déployé, l'application web est pleinement accessible via l'URL suivante : <https://briefly-frontend-5rtf.onrender.com>

Les requêtes destinées à l'API REST sont quant à elles traitées par le serveur disponible à l'adresse : <https://briefly-8t35.onrender.com>

Enfin, l'intégralité du code source est consultable sur le dépôt GitHub du projet : <https://github.com/YahyaMudallal/Briefly>

En l'état actuel, Briefly est une plateforme totalement fonctionnelle et exploitable. Elle propose un système complet d'agrégation d'articles de presse, consultables et triables soit par date de publication, soit selon un score de popularité temporelle (*hotness*). Ce score est défini par l'équation $(U - D)/A$, où U représente le nombre de votes positifs, D le nombre de votes négatifs et A l'âge de l'article calculé en heures. Ce tri peut être appliqué par ordre croissant ou décroissant selon les préférences de l'utilisateur. De plus, la composante sociale de l'application, matérialisée par la possibilité de publier des commentaires sous les articles, est entièrement opérationnelle et répond aux exigences fixées initialement.

Néanmoins, en raison de contraintes de temps, certains aspects secondaires de l'application n'ont pas pu être pleinement finalisés. C'est notamment le cas de la page de profil de l'utilisateur, qui se limite actuellement à l'affichage des données d'identification (nom, prénom et email) sans offrir la possibilité de les éditer, ni de lister l'historique des articles pour lesquels l'utilisateur a voté.